

Objectif

Savoir réaliser et mettre en place une architecture REST complète en Java EE. Ce projet couvre la manipulation de ressources complexes, l'intégration de logique métier, la gestion fine des codes de retour HTTP, le support multi-format (JSON/XML) et la sécurisation par token.

Contexte : Le projet EcoDrop

L'entreprise `EcoDrop` souhaite mettre à disposition des entreprises tierces et des citoyens une API de gestion de collecte de déchets spécifiques (batteries, textiles, électronique). L'application doit permettre de gérer les points de collecte, les types de déchets acceptés et les dépôts effectués par les utilisateurs.

L'entreprise compte sur vous pour développer son service REST offrant différentes fonctionnalités à ses usagers.

1 Partie A

1.1 Le Catalogue de Déchets

On s'intéresse tout d'abord à la table `WasteType(id, nom, pointsPerKilo)` contenant les types de déchets traités par l'entreprise

- Créer le DAO et éventuellement le DTO pour les types de déchets.
- `GET /waste-types` : Liste tous les types de déchets disponibles. Ce endpoint doit supporter `application/json` et `application/xml`.
- `GET /waste-types/id` : Détails d'un type spécifique. Renvoie 404 si l'ID n'existe pas.
- `POST /waste-types` : Ajoute un nouveau type.
- `PUT /waste-types/id` : Mise à jour complète d'un type.
- `DELETE /waste-types/id` : Supprime un type. Renvoie 409 Conflict si le type est utilisé dans la table des dépôts.

1.2 Points de Collecte

On ajoute maintenant les tables `CollectionPoint(id, adresse, capaciteMax)` contenant la liste des points de collecte et `accepts(#pointid, #wastetypeid)` indiquant quel point collecte utiliser pour quel(s) déchet(s)

- `GET /points` : Liste tous les points de collecte.
- `GET /points/id` : Détails d'un point incluant la liste imbriquée des `WasteType` acceptés.
- `PUT` et `PATCH /points/id` : Modification partielle (ex : adresse).
- `DELETE /points/id/clear` : Vide (met `collecte = true`) tous les dépôts réalisés sur un point de collecte. le calcul du "taux de remplissage" (Partie 2) pour ce point doit retomber à 0%. C'est l'action typique effectuée par un agent de collecte après avoir vidé physiquement la borne. L'utilisateur conserve ses lignes de dépôts en base pour le calcul de son score (Leaderboard)

1.3 Tests avec Bruno

- L'application doit isoler les requêtes SQL dans des **DAO** et la connexion via un objet externalisant la connexion (DS).
- Un test de chaque endpoint doit figurer dans une collection **BRUNO** (incluant les cas d'erreur).

2 Partie B

2.1 Dépôts et logique métier

On gère maintenant les tables `Deposit(id, #userId, #pointId, #wasteTypeId, poids, datedepot, collecte)` qui indique qui a déposé quoi, où, et en quelle quantité, avec un booléen qui indique si ce dépôt a été collecté (enlevé ou pas) et `users(id, login, password, role)` listant les utilisateurs, avec 2 rôles possibles USER ou ADMIN

- `GET /deposits` : fournit toutes les informations sur les dépôts (nom du déchet et adresse du point).
- `POST /deposits` : Enregistre un dépôt. Renvoie une erreur (400 Bad Request ou 403 Forbidden) si le poids est négatif ou si le point saturé.
- `GET /deposits/id` : Détails d'un dépôt spécifique.
- `GET /users/leaderboard` : Renvoie les 10 meilleurs recycleurs : $\text{score} = \sum(\text{poids} \times \text{pointsPerKilo})$.
- On peut maintenant rajouter au point de collecte `GET /points/id/status` : Renvoie identifiant et adresse du point avec le taux de remplissage dynamique `remplissage` en pourcentage, ainsi qu'une clé booléenne `full` qui indique si la capacité dépasse 100%. Le calcul du remplissage ne doit se faire que sur les dépôts où `collecte = false` non encore collecté)
- `GET /points/overloaded` : Liste les points dont l'occupation est > 80%.
- `PUT` et `PATCH` Sur `/users` et sur `deposit` pour les modifications mineures.
- Renforcez votre collection BRUNO avec ces différents endpoints
- Ecrire un programme java `import.java` qui lit un CSV et importe les dépôts décrits dans le fichier.

3 Partie C

Sécurisation et rôles

On s'occupe enfin de la table `User(id, login, password, role)` et de la protection des accès.

- **Obtention du Token** : Un endpoint `GET /auth/token` (sécurisé par Basic Auth) permet de s'authentifier et de récupérer un jeton (APIToken).
- **Accès Public** : Tous les endpoints en `GET` sont publics et ne nécessitent aucune protection (consultation libre du catalogue et des points).
- **Accès Protégé (Token)** : Toutes les opérations de modification (`POST`, `PUT`, `PATCH`, `DELETE`) nécessitent obligatoirement la présence du token valide dans le header de la requête.
- **Droits d'administration (Rôles)** :
 - Le rôle USER (ex : paul/paul) peut effectuer des dépôts (`POST /deposits`).
 - Le rôle ADMIN (ex : pierre/pierre) est le seul autorisé à utiliser les méthodes `DELETE` ainsi que l'accès aux statistiques de surcharge (`GET /points/overloaded`).

Travail à réaliser

- L'application doit isoler les requêtes SQL dans des **DAO** et la connexion via un objet externalisant la connexion (DS).

- Un test de chaque endpoint doit figurer dans une collection **BRUNO** (incluant les cas d'erreur).
- Un fichier Markdown expliquant le schéma de la base et les requêtes SQL complexes doit être fourni. Il contiendra aussi une description de l'API avec tous les codes de retour.

Conseils pour les tests (BRUNO)

Utilisez le "Runner" de Bruno pour créer des scénarios :

1. Récupérer un token.
2. Créer un type de déchet (POST).
3. Tenter de le supprimer sans token (vérifier le 401).
4. Le supprimer avec token (vérifier le 204 ou 200).